

Tradexy Deployment Guide

Last Updated: February 17, 2026 **Deployment Method:** Docker Image Registry (ghcr.io) **Stack:** PHP 8.4-FPM · Nginx · PostgreSQL 16 · Docker · GitHub Actions

Table of Contents

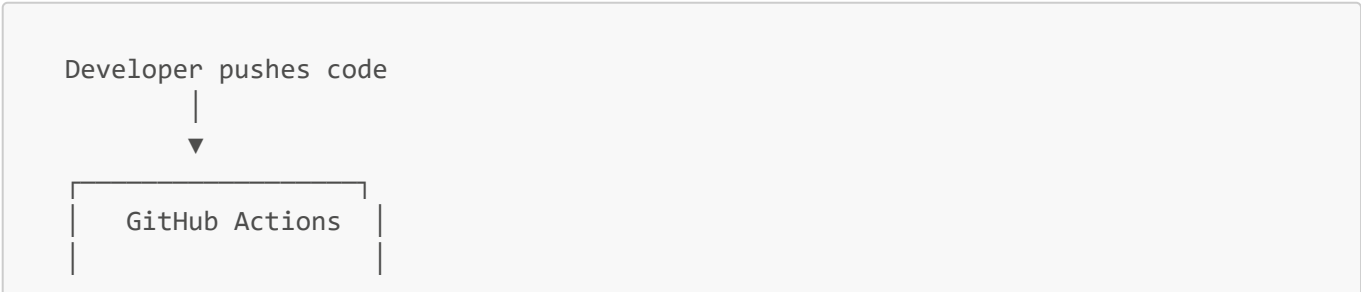
- [Architecture Overview](#)
- [Prerequisites](#)
- [How It Works](#)
- [Local Development](#)
- [One-Time Server Setup](#)
- [One-Time GitHub Setup](#)
- [CI/CD Pipeline](#)
- [Manual Deployment](#)
- [SSL & Reverse Proxy](#)
- [Database Management](#)
- [Health Checks & Monitoring](#)
- [Log Management](#)
- [Security Checklist](#)
- [Rollback](#)
- [Troubleshooting](#)
- [File Reference](#)

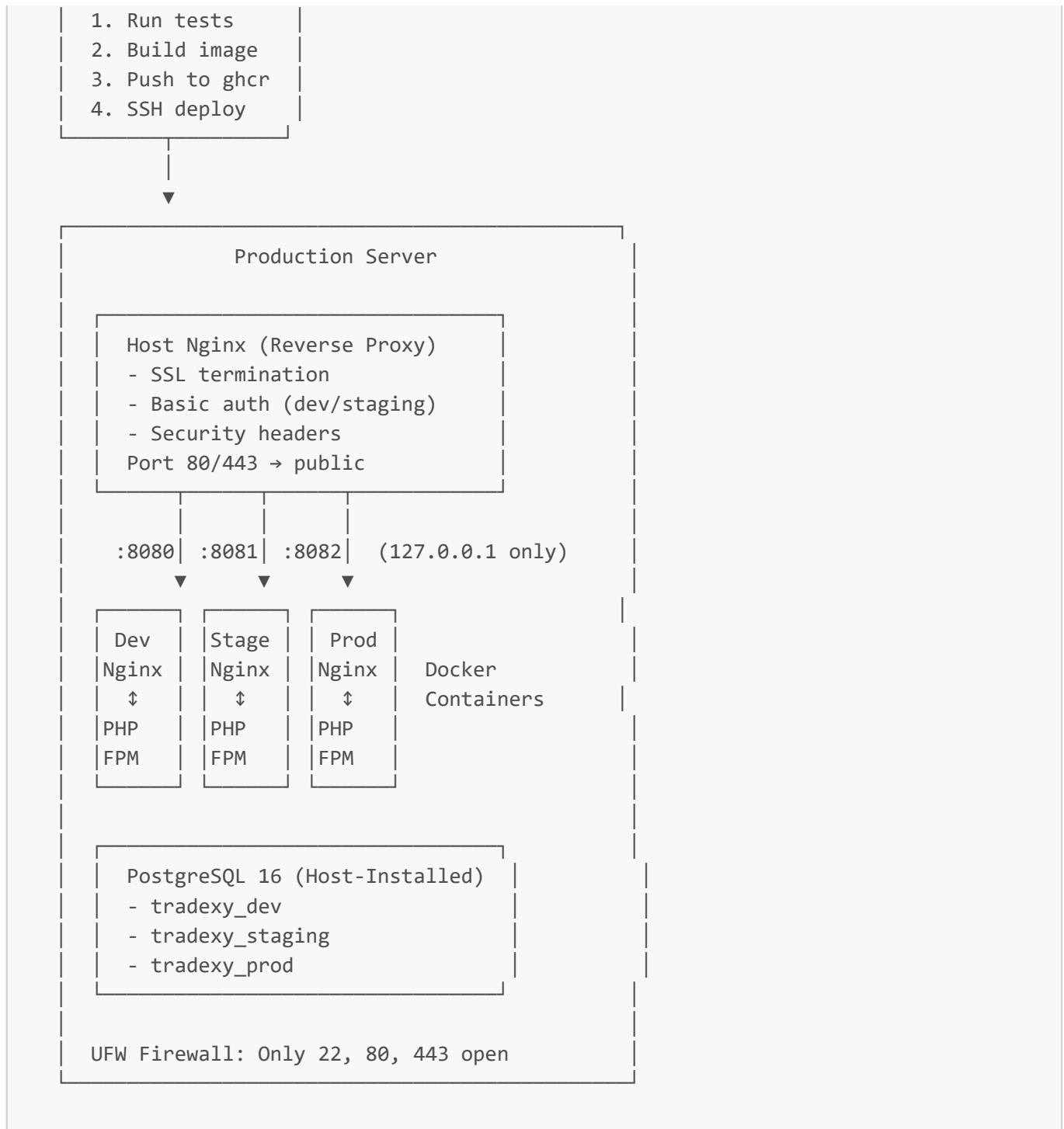
Architecture Overview

Environments

Environment	Branch	Image Tag	Internal Port	External Access
Local	any	built locally	8000	http://localhost:8000
Dev	dev	ghcr.io/...:dev	127.0.0.1:8080	Password protected
Staging	staging	ghcr.io/...:staging	127.0.0.1:8081	Password protected
Production	main	ghcr.io/...:main	127.0.0.1:8082	Public via SSL

Architecture Diagram





What Lives on the Server (Per Environment)

```

/var/www/tradexy-dev/
├── docker-compose.yml
├── default.conf
└── .env

```

- ← Just 3 files!
- ← References ghcr.io image
- ← Nginx config
- ← Environment variables (chmod 600)

No source code. No vendor/. No git repo.

Prerequisites

Requirement	Version	Purpose
Docker	>= 24.0	Container runtime (server)
Docker Compose	>= 2.20	Multi-container orchestration
Git	>= 2.40	Version control (local only)
PHP	8.4+	Local development only
PostgreSQL	16	Database (host-installed)
Nginx	latest	Host reverse proxy
Certbot	latest	SSL certificates
GitHub Account	-	CI/CD and container registry

How It Works

Previous Approach (Option 1 — Deprecated)

Push → GitHub Actions → SSH → `git pull` → `composer install` → `docker build` → up

- ✗ Full source code on server
- ✗ Builds images on server (high CPU)
- ✗ Docker build cache grows endlessly
- ✗ 3 full git repos = ~600MB+ wasted

Current Approach (Option 4 — Image Registry)

Push → GitHub Actions → Build image → Push to ghcr.io → SSH → `docker pull` → up

- ✓ No source code on server
- ✓ Build happens on GitHub (free)
- ✓ No build cache on server
- ✓ 3 tiny directories = ~12KB total
- ✓ Deploy in ~30 seconds

Local Development

Start All Services

```
docker-compose up -d
```

First-Time Setup

```
docker-compose exec app composer install
docker-compose exec app php artisan key:generate
docker-compose exec app php artisan migrate
docker-compose exec app php artisan storage:link
docker-compose exec app php artisan db:seed    # Optional
```

Daily Commands

```
# Start
docker-compose up -d

# Stop
docker-compose down

# Run tests
docker-compose exec app php artisan test

# Run a specific test
docker-compose exec app php artisan test --filter=TradeTest

# Artisan commands
docker-compose exec app php artisan make:model Trade -mfc
docker-compose exec app php artisan migrate:fresh --seed

# Composer
docker-compose exec app composer require package/name
```

Access

Service	URL
App	http://localhost:8000
PostgreSQL	localhost:5432

Reset Everything

```
docker-compose down -v --rmi all
docker-compose up -d --build
```

One-Time Server Setup

Run these commands **once** when setting up a new server.

Step 1: Create Deploy User

```
ssh root@your-server-ip

# Create user
adduser deploy --disabled-password
usermod -aG docker deploy

# Setup SSH for deploy user
mkdir -p /home/deploy/.ssh
cp /root/.ssh/authorized_keys /home/deploy/.ssh/
chown -R deploy:deploy /home/deploy/.ssh
chmod 700 /home/deploy/.ssh
chmod 600 /home/deploy/.ssh/authorized_keys
```

Step 2: Install Docker (If Not Installed)

```
curl -fsSL https://get.docker.com | sh
systemctl enable docker
systemctl start docker
```

Step 3: Install PostgreSQL 16

```
apt install postgresql-16 -y
systemctl enable postgresql
systemctl start postgresql

# Create databases and user
sudo -u postgres psql << 'SQL'
CREATE USER tradexy_user WITH PASSWORD 'YOUR_STRONG_PASSWORD_HERE';
CREATE DATABASE tradexy_dev OWNER tradexy_user;
CREATE DATABASE tradexy_staging OWNER tradexy_user;
CREATE DATABASE tradexy_prod OWNER tradexy_user;
GRANT ALL PRIVILEGES ON DATABASE tradexy_dev TO tradexy_user;
GRANT ALL PRIVILEGES ON DATABASE tradexy_staging TO tradexy_user;
GRANT ALL PRIVILEGES ON DATABASE tradexy_prod TO tradexy_user;
SQL
```

Step 4: Install Host Nginx

```
apt install nginx apache2-utils -y
systemctl enable nginx
```

Step 5: Create Project Directories

```
mkdir -p /var/www/tradexy-dev
mkdir -p /var/www/tradexy-staging
mkdir -p /var/www/tradexy-prod
chown -R deploy:deploy /var/www/tradexy-*
```

Step 6: Create Nginx Config (All Environments)

```
cat > /var/www/tradexy-dev/default.conf << 'EOF'
server {
    listen 80;
    index index.php index.html;
    root /var/www/public;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ /\.php$ {
        fastcgi_pass app:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~ /\.ht {
        deny all;
    }
}
EOF

cp /var/www/tradexy-dev/default.conf /var/www/tradexy-staging/default.conf
cp /var/www/tradexy-dev/default.conf /var/www/tradexy-prod/default.conf
```

Step 7: Create Docker Compose Files

Replace **YOUR_GITHUB_USERNAME/YOUR_REPO_NAME** with your actual values.

```
# — DEV —
cat > /var/www/tradexy-dev/docker-compose.yml << 'EOF'
services:
  app:
    image: ghcr.io/YOUR_GITHUB_USERNAME/YOUR_REPO_NAME:dev
    container_name: tradexy_app_dev
    restart: unless-stopped
    working_dir: /var/www
    volumes:
      - ./env:/var/www/.env:ro
      - app_storage:/var/www/storage
```

```

networks:
  - tradexy_dev
healthcheck:
  test: ["CMD-SHELL", "php-fpm-healthcheck || exit 1"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 30s

nginx:
  image: nginx:alpine
  container_name: tradexy_nginx_dev
  restart: unless-stopped
  ports:
    - "127.0.0.1:8080:80"
  volumes:
    - ./default.conf:/etc/nginx/conf.d/default.conf:ro
  depends_on:
    app:
      condition: service_healthy
  networks:
    - tradexy_dev

networks:
  tradexy_dev:

volumes:
  app_storage:
EOF

# — STAGING —
cat > /var/www/tradexy-staging/docker-compose.yml << 'EOF'
services:
  app:
    image: ghcr.io/YOUR_GITHUB_USERNAME/YOUR_REPO_NAME:staging
    container_name: tradexy_app_staging
    restart: unless-stopped
    working_dir: /var/www
    volumes:
      - ./env:/var/www/.env:ro
      - app_storage:/var/www/storage
    networks:
      - tradexy_staging
    healthcheck:
      test: ["CMD-SHELL", "php-fpm-healthcheck || exit 1"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 30s

  nginx:
    image: nginx:alpine
    container_name: tradexy_nginx_staging
    restart: unless-stopped

```

```

    ports:
      - "127.0.0.1:8081:80"
    volumes:
      - ./default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      app:
        condition: service_healthy
    networks:
      - tradexy_staging

networks:
  tradexy_staging:

volumes:
  app_storage:
EOF

# — PRODUCTION —
cat > /var/www/tradexy-prod/docker-compose.yml << 'EOF'
services:
  app:
    image: ghcr.io/YOUR_GITHUB_USERNAME/YOUR_REPO_NAME:main
    container_name: tradexy_app_prod
    restart: unless-stopped
    working_dir: /var/www
    volumes:
      - ../.env:/var/www/.env:ro
      - app_storage:/var/www/storage
    networks:
      - tradexy_prod
    healthcheck:
      test: ["CMD-SHELL", "php-fpm-healthcheck || exit 1"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 30s

  nginx:
    image: nginx:alpine
    container_name: tradexy_nginx_prod
    restart: unless-stopped
    ports:
      - "127.0.0.1:8082:80"
    volumes:
      - ./default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      app:
        condition: service_healthy
    networks:
      - tradexy_prod

networks:
  tradexy_prod:

```



```
volumes:
  app_storage:
EOF
```

Step 8: Create .env Files

```
# Create .env for each environment
# Use different APP_KEY, DB_DATABASE, and APP_URL for each

cat > /var/www/tradexy-dev/.env << 'EOF'
APP_NAME=Tradexy
APP_ENV=dev
APP_KEY=
APP_DEBUG=true
APP_URL=https://dev.tradexy.com

DB_CONNECTION=pgsql
DB_HOST=host.docker.internal
DB_PORT=5432
DB_DATABASE=tradexy_dev
DB_USERNAME=tradexy_user
DB_PASSWORD=YOUR_STRONG_PASSWORD_HERE

LOG_CHANNEL=stack
LOG_LEVEL=debug
EOF

cat > /var/www/tradexy-staging/.env << 'EOF'
APP_NAME=Tradexy
APP_ENV=staging
APP_KEY=
APP_DEBUG=false
APP_URL=https://staging.tradexy.com

DB_CONNECTION=pgsql
DB_HOST=host.docker.internal
DB_PORT=5432
DB_DATABASE=tradexy_staging
DB_USERNAME=tradexy_user
DB_PASSWORD=YOUR_STRONG_PASSWORD_HERE

LOG_CHANNEL=stack
LOG_LEVEL=warning
EOF

cat > /var/www/tradexy-prod/.env << 'EOF'
APP_NAME=Tradexy
APP_ENV=production
APP_KEY=
APP_DEBUG=false
APP_URL=https://app.tradexy.com
```

```

DB_CONNECTION=pgsql
DB_HOST=host.docker.internal
DB_PORT=5432
DB_DATABASE=tradexy_prod
DB_USERNAME=tradexy_user
DB_PASSWORD=YOUR_STRONG_PASSWORD_HERE

LOG_CHANNEL=stack
LOG_LEVEL=error
EOF

# Generate APP_KEY for each
for dir in /var/www/tradexy-dev /var/www/tradexy-staging /var/www/tradexy-prod; do
    NEW_KEY="base64:$(openssl rand -base64 32)"
    sed -i "s|^APP_KEY=.*|APP_KEY=$NEW_KEY|" $dir/.env
    echo "✅ APP_KEY generated for $dir"
done

# Lock down permissions
chmod 600 /var/www/tradexy-dev/.env
chmod 600 /var/www/tradexy-staging/.env
chmod 600 /var/www/tradexy-prod/.env
chown deploy:deploy /var/www/tradexy-*/.env

```

Note on DB_HOST: Use `host.docker.internal` if Docker supports it on your server (Docker 20.10+). Otherwise use the server's private IP or `172.17.0.1` (Docker bridge gateway). Test with:

```
docker run --rm alpine ping host.docker.internal
```

If it doesn't resolve, use `172.17.0.1` and ensure PostgreSQL listens on that interface.

Step 9: Setup Firewall

```

ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable
ufw status verbose

```

Step 10: Setup Host Reverse Proxy

```

# Create password for dev/staging access
htpasswd -c /etc/nginx/.htpasswd dev
# Enter a strong password when prompted

```

```
cat > /etc/nginx/sites-available/tradexy << 'NGINX'
# =====
# DEV – Password Protected
# =====
server {
    listen 80;
    server_name dev.tradexy.com;

    auth_basic "Dev Environment";
    auth_basic_user_file /etc/nginx/.htpasswd;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# =====
# STAGING – Password Protected
# =====
server {
    listen 80;
    server_name staging.tradexy.com;

    auth_basic "Staging Environment";
    auth_basic_user_file /etc/nginx/.htpasswd;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    location / {
        proxy_pass http://127.0.0.1:8081;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# =====
# PRODUCTION – Public, SSL Ready
# =====
server {
```

```

listen 80;
server_name app.tradexy.com;

add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;

location / {
    proxy_pass http://127.0.0.1:8082;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
NGINX

ln -sf /etc/nginx/sites-available/tradexy /etc/nginx/sites-enabled/
rm -f /etc/nginx/sites-enabled/default
nginx -t && systemctl restart nginx

```

Step 11: Install SSL Certificates

```

apt install certbot python3-certbot-nginx -y
certbot --nginx -d app.tradexy.com -d staging.tradexy.com -d dev.tradexy.com

# Auto-renewal (certbot adds this automatically, verify with):
systemctl status certbot.timer

```

Step 12: Verify Server Structure

```

echo "=== Directory Structure ==="
for dir in /var/www/tradexy-dev /var/www/tradexy-staging /var/www/tradexy-prod; do
    echo ""
    echo "--- $dir ---"
    ls -la $dir/
done

echo ""
echo "=== Disk Usage ==="
df -h /

echo ""
echo "=== Firewall ==="
ufw status

```

```
echo ""
echo "=== Nginx ==="
nginx -t
```

Expected output per directory:

```
--- /var/www/tradexy-dev ---
-rw----- 1 deploy deploy 512 Feb 17 .env
-rw-r--r-- 1 deploy deploy 650 Feb 17 docker-compose.yml
-rw-r--r-- 1 deploy deploy 350 Feb 17 default.conf
```

One-Time GitHub Setup

Step 1: Create Personal Access Token

- 1. Go to **GitHub** → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)**
- 2. Click **Generate new token (classic)**
- 3. **Name:** tradexy-server-deploy
- 4. **Expiration:** No expiration (or 1 year)
- 5. **Scopes:** Check **read:packages** and **write:packages**
- 6. Click **Generate token** → **Copy it immediately**

Step 2: Add Repository Secrets

Go to **repo** → **Settings** → **Secrets and variables** → **Actions** → **New repository secret:**

Secret Name	Value
SERVER_HOST	Your server IP address
SERVER_USERNAME	deploy
SERVER_SSH_KEY	Private SSH key (see below)
GH_PAT	Personal access token from Step 1

Step 3: Generate SSH Key for CI/CD

```
# Run in PowerShell on your local machine
ssh-keygen -t ed25519 -C "github-actions-deploy" -f
"$env:USERPROFILE\.ssh\tradexy_deploy" -N ''

# Display private key (add this as SERVER_SSH_KEY secret)
Get-Content "$env:USERPROFILE\.ssh\tradexy_deploy"

# Display public key (add this to server)
Get-Content "$env:USERPROFILE\.ssh\tradexy_deploy.pub"
```

Add the public key to the server:

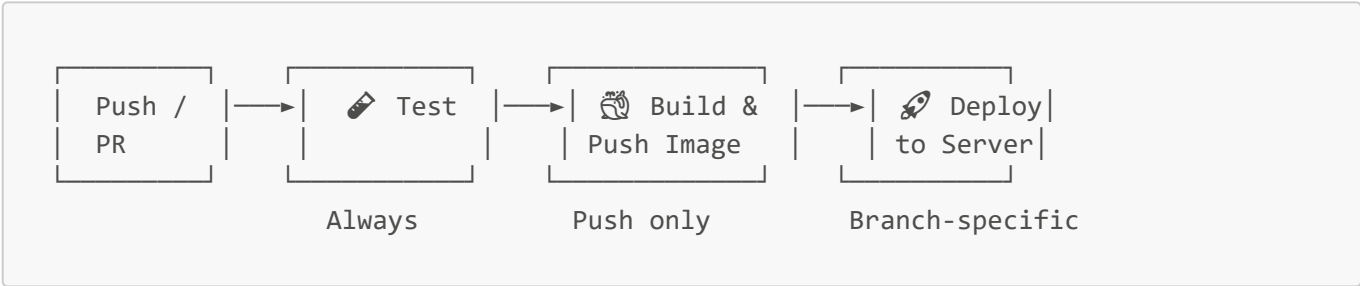
```
# On server
echo "PASTE_PUBLIC_KEY_HERE" >> /home/deploy/.ssh/authorized_keys
```

Step 4: Enable Package Permissions

- 1. Go to **repo** → **Settings** → **Actions** → **General**
- 2. Scroll to **Workflow permissions**
- 3. Select **Read and write permissions**
- 4. Check **Allow GitHub Actions to create and approve pull requests**
- 5. Save

CI/CD Pipeline

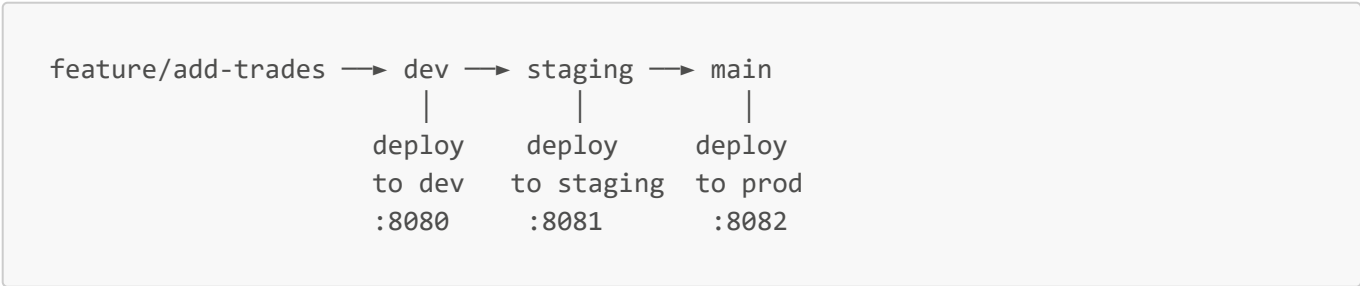
Pipeline Flow



Pipeline Jobs

Job	Trigger	What It Does
test	All pushes & PRs	Run test suite against PostgreSQL
build	Push only (not PR)	Build Docker image, push to ghcr.io
deploy-dev	Push to dev	Pull image on server, restart containers
deploy-staging	Push to staging	Pull image, cache config, restart
deploy-production	Push to main	Pull image, cache config, restart

Branch Workflow



What Happens During Deploy

```
1. SSH into server as 'deploy' user
2. cd /var/www/tradexy-{env}
3. Verify .env exists
4. Ensure APP_KEY is set
5. docker login to ghcr.io (using GH_PAT)
6. docker pull latest image
7. docker-compose down
8. docker-compose up -d
9. Wait for health check (15s)
10. php artisan migrate --force
11. php artisan storage:link
12. Cache config/routes/views (staging & prod only)
13. Set file permissions
14. Prune old Docker images (older than 7 days)
15. docker logout (remove credentials)
```

Security Measures in Pipeline

Measure	Purpose
Secrets passed via <code>envs</code>	Prevents interpolation leaks in logs
<code>docker logout</code> after deploy	Removes registry credentials from server
<code>deploy</code> user (not root)	Principle of least privilege
Image prune after deploy	Prevents disk usage growth
<code>.env</code> mounted as <code>:ro</code>	Container can't modify secrets
Health check before nginx	Prevents 502 errors during deploy

Manual Deployment

For emergency deployments or if CI/CD is down.

From Server

```
ssh deploy@your-server-ip

# Login to registry
echo "YOUR_GH_PAT" | docker login ghcr.io -u YOUR_GITHUB_USERNAME --password-stdin

# Deploy dev
cd /var/www/tradexy-dev
docker pull ghcr.io/YOUR_GITHUB_USERNAME/YOUR_REPO_NAME:dev
docker-compose down && docker-compose up -d
sleep 15
```

```
docker-compose exec -T app php artisan migrate --force
docker-compose exec -T app php artisan storage:link || true

# Deploy staging
cd /var/www/tradexy-staging
docker pull ghcr.io/YOUR_GITHUB_USERNAME/YOUR_REPO_NAME:staging
docker-compose down && docker-compose up -d
sleep 15
docker-compose exec -T app php artisan migrate --force
docker-compose exec -T app php artisan config:cache
docker-compose exec -T app php artisan route:cache
docker-compose exec -T app php artisan view:cache

# Deploy production
cd /var/www/tradexy-prod
docker pull ghcr.io/YOUR_GITHUB_USERNAME/YOUR_REPO_NAME:main
docker-compose down && docker-compose up -d
sleep 15
docker-compose exec -T app php artisan migrate --force
docker-compose exec -T app php artisan config:cache
docker-compose exec -T app php artisan route:cache
docker-compose exec -T app php artisan view:cache

# Cleanup
docker image prune -a --force --filter "until=168h"
docker logout ghcr.io
```

Using deploy.sh Script

Copy `deploy.sh` to the server:

```
scp deploy.sh deploy@your-server-ip:/usr/local/bin/tradexy-deploy
ssh deploy@your-server-ip "chmod +x /usr/local/bin/tradexy-deploy"
```

Then run:

```
ssh deploy@your-server-ip
tradexy-deploy dev
tradexy-deploy staging
tradexy-deploy production
```

SSL & Reverse Proxy

How It Works


```

Internet → Port 443 (SSL) → Host Nginx → 127.0.0.1:808x → Docker Nginx → PHP-FPM
                        |
                        Basic Auth
                        (dev & staging)

```

Verify SSL

```

# Check certificate status
certbot certificates

# Test SSL
curl -I https://app.tradexy.com
curl -I https://staging.tradexy.com
curl -I https://dev.tradexy.com

```

Renew SSL Manually

```

certbot renew --dry-run    # Test first
certbot renew              # Actually renew

```

Add/Change Basic Auth Password

```

# Add new user
htpasswd /etc/nginx/.htpasswd newuser

# Change existing password
htpasswd /etc/nginx/.htpasswd dev

# Remove user
htpasswd -D /etc/nginx/.htpasswd olduser

# Restart nginx
systemctl restart nginx

```

Database Management

Backup

```

# Manual backup
pg_dump -U tradexy_user -h 127.0.0.1 tradexy_prod > backup_$(date
+%Y%m%d_%H%M%S).sql

```

```
# Compressed backup
pg_dump -U tradexy_user -h 127.0.0.1 tradexy_prod | gzip > backup_$(date +%Y%m%d).sql.gz
```

Restore

```
# From SQL file
psql -U tradexy_user -h 127.0.0.1 tradexy_prod < backup_file.sql

# From compressed file
gunzip -c backup_file.sql.gz | psql -U tradexy_user -h 127.0.0.1 tradexy_prod
```

Automated Daily Backups (Cron)

```
# Create backup directory
mkdir -p /var/backups/tradexy

# Add to crontab: crontab -e
0 2 * * * pg_dump -U tradexy_user -h 127.0.0.1 tradexy_prod | gzip >
/var/backups/tradexy/prod_$(date +%Y%m%d).sql.gz && find /var/backups/tradexy -
mtime +30 -delete
```

Migration Rollback

```
# Rollback last batch
docker-compose exec -T app php artisan migrate:rollback

# Rollback specific number of steps
docker-compose exec -T app php artisan migrate:rollback --step=2

# Check migration status
docker-compose exec -T app php artisan migrate:status
```

Health Checks & Monitoring

Quick Status

```
# All containers at a glance
docker ps --filter "name=tradexy" --format "table
{{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Expected output:

NAMES	STATUS	PORTS
tradexy_nginx_prod	Up 2 hours	127.0.0.1:8082->80/tcp
tradexy_app_prod	Up 2 hours (healthy)	9000/tcp
tradexy_nginx_staging	Up 2 hours	127.0.0.1:8081->80/tcp
tradexy_app_staging	Up 2 hours (healthy)	9000/tcp
tradexy_nginx_dev	Up 2 hours	127.0.0.1:8080->80/tcp
tradexy_app_dev	Up 2 hours (healthy)	9000/tcp

Health Check Endpoints

```
# Check HTTP status code
curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:8080
curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:8081
curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:8082
```

Application Status

```
# Laravel info
cd /var/www/tradexy-prod
docker-compose exec -T app php artisan about

# Database connectivity
docker-compose exec -T app php artisan db:show

# Check routes are registered
docker-compose exec -T app php artisan route:list --compact
```

Disk Usage Monitoring

```
# Overall disk
df -h /

# Docker disk usage
docker system df

# Project directories
du -sh /var/www/tradexy-*

# Docker volumes
docker volume ls
docker system df -v
```

Log Management

Application Logs

```
# Laravel logs
cd /var/www/tradexy-prod
docker-compose exec -T app tail -f /var/www/storage/logs/laravel.log
docker-compose exec -T app tail -100 /var/www/storage/logs/laravel.log
```

Container Logs

```
# All services
docker-compose logs -f

# Specific service
docker-compose logs -f app
docker-compose logs -f nginx

# Last 50 lines
docker-compose logs --tail 50 app
```

Host Nginx Logs

```
tail -f /var/log/nginx/access.log
tail -f /var/log/nginx/error.log
```

Log Rotation

```
cat > /etc/logrotate.d/tradexy << 'EOF'
/var/www/tradexy-*/storage/logs/*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
}
EOF
```

Clean Old Logs

```
# Clean journal logs
journalctl --vacuum-time=7d
```

```
# Clean Docker container logs
truncate -s 0 $(docker inspect --format='{{.LogPath}}' tradexy_app_prod)
```

Security Checklist

Initial Setup

- ☐ Deploy user created (not using root)
- ☐ SSH key authentication configured
- ☐ Password authentication disabled in `/etc/ssh/sshd_config`
- ☐ UFW firewall enabled (only 22, 80, 443)
- ☐ Docker ports bound to `127.0.0.1`
- ☐ `.env` files have `chmod 600`
- ☐ `.env` mounted as read-only (`:ro`) in containers
- ☐ Basic auth on dev/staging environments
- ☐ SSL certificates installed
- ☐ Unique `APP_KEY` per environment
- ☐ `APP_DEBUG=false` in staging and production
- ☐ Unique database passwords per environment
- ☐ GitHub secrets configured (not hardcoded)
- ☐ `GH_PAT` has minimal scope (`read:packages`, `write:packages` only)
- ☐ `.env` is in `.gitignore`
- ☐ `.dockerignore` excludes sensitive files

Ongoing Maintenance

- ☐ Monitor disk usage weekly
- ☐ Review GitHub Actions logs for failures
- ☐ Rotate database passwords quarterly
- ☐ Renew SSL certificates (auto via certbot)
- ☐ Run `composer audit` monthly for vulnerabilities
- ☐ Update Docker images monthly
- ☐ Review server access logs for suspicious activity
- ☐ Test database backup restores monthly
- ☐ Keep server packages updated: `apt update && apt upgrade`

Security Headers (Applied by Host Nginx)

Header	Value	Purpose
<code>X-Frame-Options</code>	<code>SAMEORIGIN</code>	Prevent clickjacking
<code>X-Content-Type-Options</code>	<code>nosniff</code>	Prevent MIME sniffing
<code>X-XSS-Protection</code>	<code>1; mode=block</code>	XSS protection
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	Control referrer info

Header	Value	Purpose
Strict-Transport-Security	max-age=31536000	Force HTTPS (prod only)

Rollback

Rollback to Previous Image

```
# Find available image tags
docker images --filter "reference=ghcr.io/*/trading-journal-v2" --format "{{.Tag}}
{{.CreatedAt}}"

# Or check GitHub Packages for available tags
# Go to: github.com/YOUR_USERNAME/trading-journal-v2/pkgs/container/trading-
journal-v2

# Rollback to specific SHA
cd /var/www/tradexy-prod
docker pull ghcr.io/YOUR_USERNAME/trading-journal-v2:abc1234
# Edit docker-compose.yml to use the specific tag
# Then restart
docker-compose down && docker-compose up -d
```

Rollback Database

```
# Rollback migrations
cd /var/www/tradexy-prod
docker-compose exec -T app php artisan migrate:rollback

# Or restore from backup
gunzip -c /var/backups/tradexy/prod_20260216.sql.gz | psql -U tradexy_user -h
127.0.0.1 tradexy_prod
```

Emergency: Revert to Last Known Good State

```
cd /var/www/tradexy-prod

# 1. Stop current
docker-compose down

# 2. Pull last known good image
docker pull ghcr.io/YOUR_USERNAME/trading-journal-v2:GOOD_SHA

# 3. Update docker-compose.yml with the specific tag
sed -i 's|image: ghcr.io/.*|image: ghcr.io/YOUR_USERNAME/trading-journal-
v2:GOOD_SHA|' docker-compose.yml
```

```
# 4. Restart
docker-compose up -d

# 5. Rollback migrations if needed
docker-compose exec -T app php artisan migrate:rollback
```

Troubleshooting

Container Won't Start

```
# Check logs
docker-compose logs -f app

# Check if port is in use
ss -tlnp | grep -E "808[0-2]"

# Check Docker daemon
systemctl status docker
```

502 Bad Gateway

```
# PHP-FPM might not be ready – check health
docker inspect tradexy_app_prod --format='{{.State.Health.Status}}'

# Check PHP-FPM is running inside container
docker exec tradexy_app_prod ps aux | grep php-fpm

# Check nginx can reach app
docker exec tradexy_nginx_prod ping -c 3 app
```

"Manifest Not Found" When Pulling Image

```
# Image hasn't been built yet
# Check GitHub Actions → is the build job passing?

# Verify image exists
docker manifest inspect ghcr.io/YOUR_USERNAME/trading-journal-v2:dev
```

"Unauthorized" When Pulling Image

```
# Login manually
echo "YOUR_GH_PAT" | docker login ghcr.io -u YOUR_USERNAME --password-stdin
```

```
# Check PAT has read:packages scope
# GitHub → Settings → Developer settings → Personal access tokens
```

Database Connection Refused

```
# Check PostgreSQL is running
systemctl status postgresql

# Check it listens on the right interface
ss -tlnp | grep 5432

# Test connection from container
docker exec tradexy_app_prod php -r "new
PDO('pgsql:host=host.docker.internal;dbname=tradexy_prod', 'tradexy_user',
'password');"

# If host.docker.internal doesn't work, try:
docker exec tradexy_app_prod ping 172.17.0.1
# Then update DB_HOST in .env to 172.17.0.1
```

.env Not Loaded

```
# Check mount
docker exec tradexy_app_prod cat /var/www/.env | head -5

# Check APP_KEY
docker exec tradexy_app_prod php artisan tinker --execute="echo
config('app.key');"

```

Disk Full Again

```
# Check what's using space
docker system df
du -sh /var/www/tradexy-*
du -sh /var/log/*
journalctl --disk-usage

# Clean everything
docker system prune -a --volumes --force
docker builder prune -a --force
journalctl --vacuum-time=3d
apt-get clean && apt-get autoremove -y
```

CI/CD Pipeline Fails

Error	Cause	Fix
SSH connection refused	Wrong host/key	Verify <code>SERVER_HOST</code> , <code>SERVER_SSH_KEY</code>
Permission denied	Wrong user	Ensure <code>deploy</code> user has Docker access
Build fails	Dockerfile error	Check <code>docker build</code> logs in Actions
Push to ghcr fails	Token permissions	Enable write packages in repo settings
Tests fail	Code error	Fix tests locally first

File Reference

Local Project Files

File	Purpose
<code>docker/php/Dockerfile</code>	Local dev PHP-FPM image
<code>docker/php/Dockerfile.prod</code>	Production PHP-FPM image (used by CI/CD)
<code>docker/nginx/default.conf</code>	Local Nginx config
<code>docker-compose.yml</code>	Local development (app + nginx + postgres)
<code>.dockerignore</code>	Excludes files from Docker image
<code>.github/workflows/ci-cd.yml</code>	CI/CD pipeline definition
<code>deploy.sh</code>	Manual deployment script (for server)
<code>deployment-guide.md</code>	This file

Server Files (Per Environment)

File	Purpose
<code>docker-compose.yml</code>	References ghcr.io image, binds to 127.0.0.1
<code>default.conf</code>	Nginx config for Docker container
<code>.env</code>	Environment variables (chmod 600, mounted :ro)

GitHub Secrets

Secret	Value
<code>SERVER_HOST</code>	Server IP address
<code>SERVER_USERNAME</code>	<code>deploy</code>
<code>SERVER_SSH_KEY</code>	Ed25519 private key
<code>GH_PAT</code>	Personal access token

Quick Reference Commands

```
# — STATUS —
docker ps --filter "name=tradexy"
df -h /
docker system df

# — LOGS —
cd /var/www/tradexy-prod
docker-compose logs -f
docker-compose exec -T app tail -f /var/www/storage/logs/laravel.log

# — SHELL —
docker exec -it tradexy_app_prod bash

# — ARTISAN —
docker-compose exec -T app php artisan migrate:status
docker-compose exec -T app php artisan tinker

# — RESTART —
docker-compose restart
docker-compose down && docker-compose up -d

# — MANUAL DEPLOY —
tradexy-deploy production

# — CLEANUP —
docker image prune -a --force --filter "until=168h"
docker system prune -a --volumes --force
```